SYNOPSIS REPORT

Title of the Project:
Snake and Ladder Game Using Python

Submitted by:
Name: _____Chandan singh_____
Course / Roll No: _____20_____
Department: _____B.tech(AIML)_____
College / University: _____D.Y. Patil University Ambi_____

Table of Contents

S. No	
1	Introduction
2	Methodology
2.1	Tools Used
2.2	Functions Used
2.3	Data Structures Used
2.4	Methods Implemented
2.5	System Flowchart
2.6	Architecture Diagram
3	Future Scope
4	Conclusion

1. Introduction

Games have always been a fundamental part of human entertainment and cognitive development. Among various board games, **Snake and Ladder** is one of the most popular due to its simplicity, unpredictability, and excitement. Traditionally played using a physical board and dice, this project recreates the same game using Python programming.

- Rolling a dice (1–6)
- Snakes (which pull the player backward)
- Ladders (which push the player forward)
- Turn-based gameplay for two players
- Automatic win detection

This project demonstrates several Python concepts:

- Loops (while loops for continuous play)
- Conditionals
- Random number generation
- Dictionaries for snake/ladder positions
- Modular programming using functions

2. Methodology

The development approach for this game is modular, structured, and interactive, following the standard format given in the template

The methodology involves:

1. Initializing player positions
2. Defining snakes and ladders through a dictionary structure
3. Using a continuous loop to simulate alternate turns
4. Taking user input to roll the dice

5. Using the random module to generate dice values
6. Updating player positions accordingly
7. Checking for snake bite or ladder climb
8. Checking for winning condition
9. Printing positions after each turn

The program continues until one of the players reaches or crosses position 30.

2.1 Tools Used

The following tools and technologies were used:

Software / Technologies

- Python 3.x
- VS Code
- Built-in Libraries:
    - random → For simulating dice roll

2.2 Functions Used

1. ladder_snake()

This is the central game function responsible for:

- Creating player positions
- Alternating turns
- Rolling dice
- Checking for snakes/ladders
- Displaying progress
- Declaring the winner

Key operations inside this function:

Functionality	Description
Dice rolling	random.randint(1,6) used to simulate a dice
Player movement	Add dice result to player position
Snake/Ladder check	Uses dictionary lookups to jump positions
Turn switching	turn = 1 or turn = 2 logic
Win detection	Ends loop when any player reaches 30

2.3 Data Structures Used

Dictionary

The game uses a dictionary named jump to store snake and ladder transitions:

This dictionary allows:

- Quick lookup of special positions
- Easy modification of snake/ladder locations

Integers

Used for:

- Player positions
- Dice values
- Turn selection

2.4 Methods Implemented

Although there is one core function, several logical methods (operations) are internally implemented:

1. Dice Simulation

```
roll = random.randint(1, 6)
```

Simulates real dice behavior.

2. Player Position Update

```
player1 += roll
player2 += roll
```

3. Snake/Ladder Handling

```
if player1 in jump:
    player1 = jump[player1]
```

4. Turn Control

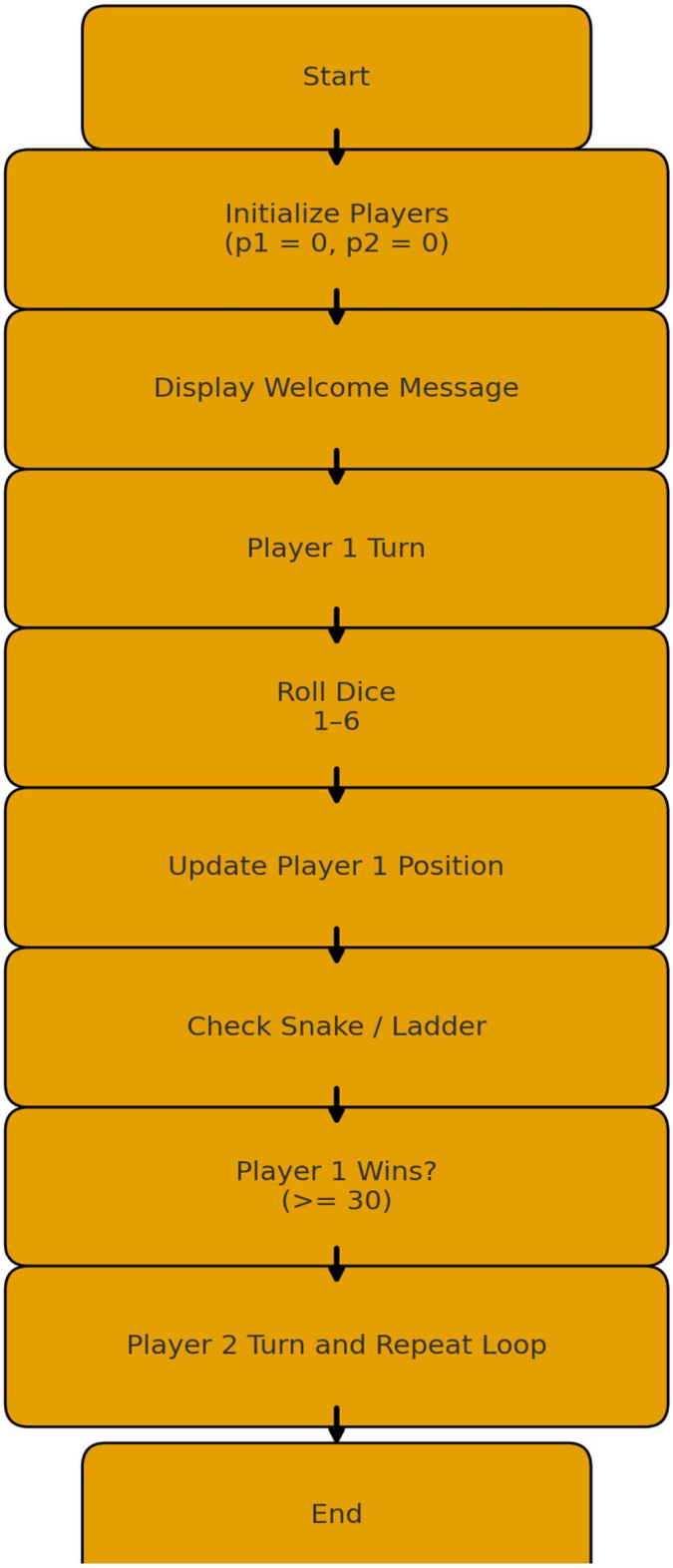
```
if turn == 1:
    turn = 2
else:
    turn = 1
```

5. Win Condition

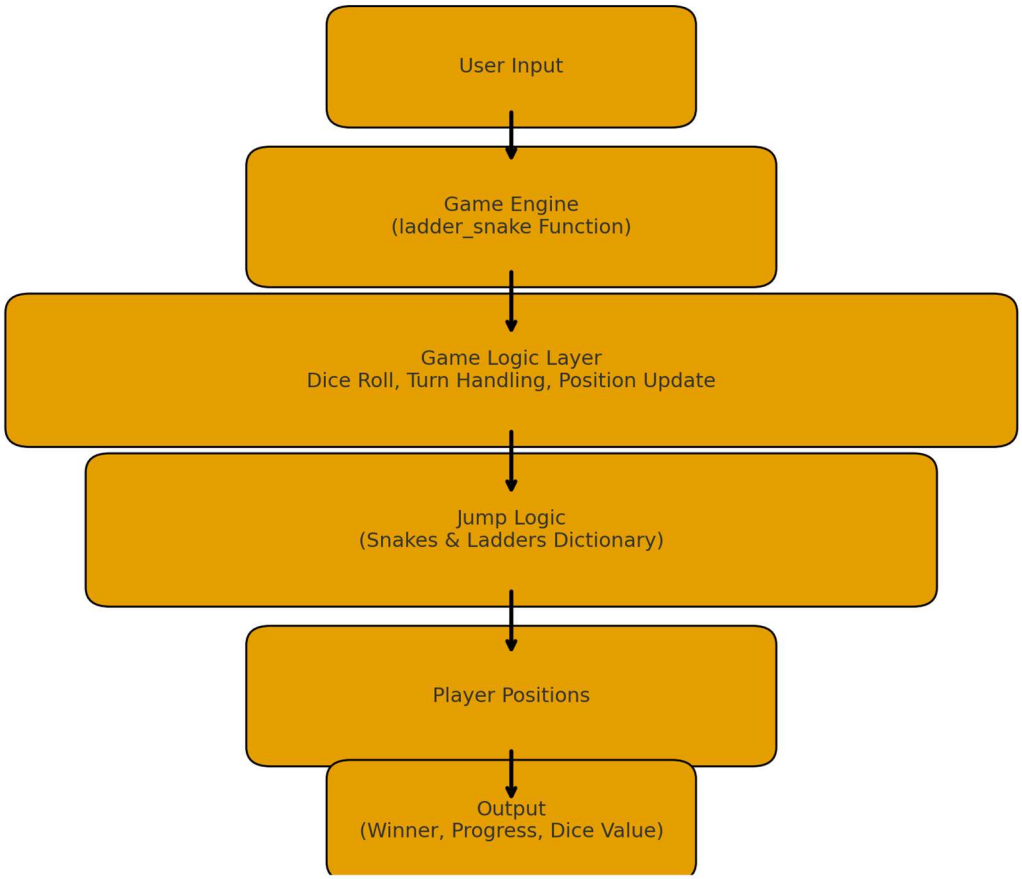
```
if player1 >= 30:
    print("Player 1 wins")
```

These internal mechanisms together form the complete game logic.

2.5 System Flowchart



2.6 Architecture Diagram



---

3. Future Scope

Technical Enhancements

- Add GUI using Tkinter or Pygame
- Show graphical board
- Add animations for snakes and ladders
- Add sound effects

Game Enhancements

- More players (3–4 players)

- Larger board (1 to 100 squares)
- Multiple snakes and ladders
- Power boosters (Skip turn, bonus turn, etc.)

Advanced Features

- Online multiplayer
- Save and load game progress
- Mobile application version

These improvements can transform the basic console game into a full-fledged board game simulation.

4. Conclusion

The Snake and Ladder Game in Python successfully demonstrates how classical board games can be recreated using simple programming constructs. The project effectively applies:

- Loops
- Conditional statements
- Randomization
- Dictionaries
- Modular programming

The program is interactive, easy to understand, and fun to play. It also provides a strong foundation for understanding game development logic in Python. With future enhancements, it has potential for GUI-based professional game development.